CORRIGÉ · PROBLÈME · ANNALES SPÉ

Décodage par l'algorithme de Viterbi

Énoncé

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Viterbi. Bob observe `Obs` $= [\text{obs}_0, \dots, \text{obs}_{N-1}]$, symboles de $\Sigma = [\![0, K-1]\!]$. $E[i][j]$: probabilité d'observer i sachant que j a été émis, ici $E = [[0.7, 0.2, 0.3], [0.2, 0.7, 0.1], [0.1, 0.1, 0.6]]$. $P[i][j]$: probabilité d'émettre j à l'instant $t{+}1$ sachant que i a été émis à l'instant t, ici $P = [[0.3, 0.2, 0.5], [0.4, 0.4, 0.2], [0.2$

Graphe : un sommet $S_{i,j}$ par symbole i et par instant j ; arcs de chaque $S_{i,j}$ vers tous les $S_{k,j+1}$; une source σ reliée à la première couche et une cible τ reliée à la dernière. Poids : $\sigma \to S_{i,0}$: $E_{\text{obs}_0,i}$; $S_{i,j} \to S_{k,j+1}$: $E_{\text{obs}_{j+1},k} P_{i,k}$; vers τ : 1. La probabilité d'un chemin est le produit des poids ; on cherche le chemin de probabilité maximale.

1. Nombre de sommets et d'arcs (sans σ, τ et leurs arcs).
2. Pour `Obs = [2,0]`, construire le graphe pondéré.
3. Nombre de chemins de σ à τ ; une exploration exhaustive est-elle envisageable ?
4. Écrire `maximumListe(liste) -> (float, int)` : maximum et plus petit indice où il est atteint.
5. Écrire `glouton(Obs, P, E, K, N) -> [int]`, qui suit à chaque étape l'arc de plus forte probabilité (on dispose de `initialiserGlouton`).
6. Complexité de l'approche gloutonne.
7. Chemin glouton sur la figure 4 (arcs $\sigma \to 0 : 0{,}6$, $\sigma \to 1 : 0{,}4$; $0 \to 0 : 0{,}5$, $0 \to 1 : 0{,}1$, $1 \to 0 : 0{,}9$, $1 \to 1 : 0{,}01$; arcs vers $\tau : 1$). Optimalité ?
8. Ramener la recherche à un plus court chemin à poids positifs ; quel algorithme ?

$T_{i,0} = E_{\text{obs}_0,i}$ et, pour $j \geqslant 1$, $T_{i,j} = \max_k T_{k,j-1} P_{k,i} E_{\text{obs}_j,i}$; `argT` garde l'indice k réalisant le maximum (-1 pour $j = 0$). On dispose de `initialiserViterbi(E, Obs0, K, N)` qui crée `T` et `argT` (K lignes, N colonnes) avec la colonne 0 remplie.

1. Écrire `construireTableauViterbi(Obs, P, E, K, N)` (de bas en haut).
2. Avec les tableaux donnés (dernière colonne de `T` : $1{,}8 \cdot 10^{-5}$, $3{,}5 \cdot 10^{-6}$, $1{,}3 \cdot 10^{-5}$; `argT` lignes : $[-1, 2, 0, 0, 2, 1, 1, 0]$, $[-1, 2, 2, 2, 2, 1, 1, 0]$, $[-1, 2, 0, 0, 2, 1, 1, 0]$), donner la séquence la plus probable.

3. Complexités temporelle et spatiale de la programmation dynamique.

Décoder l'énoncé

Ce que l'énoncé te donne.

- Graphe en N couches de K sommets, arcs complets entre deux couches consécutives.
- Probabilités dans $]0, 1]$: $-\ln$ les rend positives.

Ce qu'on te demande. Comparer exploration exhaustive, glouton et programmation dynamique pour trouver la séquence émise la plus probable.

Corrigé

Question 15

Étape 1. Q15. Sommets et arcs

Sommets : N couches (une par observation), K sommets par couche : NK sommets.

Arcs : entre deux couches consécutives, chacun des K sommets est relié aux K sommets suivants : K^2 arcs. Il y a $N-1$ paires de couches consécutives.

$$NK \text{ sommets}, \qquad (N-1)K^2 \text{ arcs}.$$

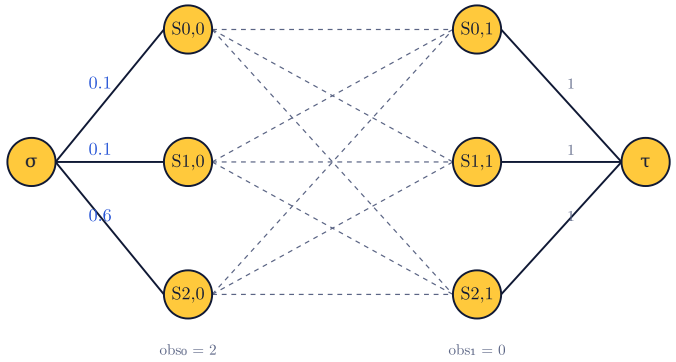
Question 16

Étape 1. Q16. Le graphe pour Obs = [2, 0]

Arcs issus de σ : $E_{\text{obs}_0,i} = E_{2,i}$, ligne 2 de E : 0,1 ; 0,1 ; 0,6.

Arcs internes $S_{i,0} \to S_{k,1}$: $E_{\text{obs}_1,k} P_{i,k} = E_{0,k} P_{i,k}$, avec la ligne 0 de E : (0,7 ; 0,2 ; 0,3).

de \{} vers	$S_{0,1}$	$S_{1,1}$	$S_{2,1}$
$S_{0,0}$	$0{,}7 \times 0{,}3 = 0{,}21$	$0{,}2 \times 0{,}2 = 0{,}04$	$0{,}3 \times 0{,}5 = 0{,}15$
$S_{1,0}$	$0{,}7 \times 0{,}4 = 0{,}28$	$0{,}2 \times 0{,}4 = 0{,}08$	$0{,}3 \times 0{,}2 = 0{,}06$
$S_{2,0}$	$0{,}7 \times 0{,}2 = 0{,}14$	$0{,}2 \times 0{,}3 = 0{,}06$	$0{,}3 \times 0{,}5 = 0{,}15$



Arcs depuis σ : $E_{2,i}$. Les neuf arcs du milieu sont pondérés par $E_{0,k} P_{i,k}$ (tableau ci-dessus).

Arcs vers τ : poids 1.

Question 17

Étape 1. Q17. Combien de chemins ?

Un chemin de σ à τ choisit exactement un sommet dans chacune des N couches, et tous les choix sont possibles (arcs complets) : il y a K choix par couche, donc

$$K^N \text{ chemins.}$$

C'est exponentiel en N : pour $K = 3$ et un message de seulement $N = 100$ symboles, $3^{100} \approx 5 \times 10^{47}$ chemins. Une exploration exhaustive est **impossible** en pratique.

Question 18

Étape 1. Q18. maximumListe

```
def maximumListe(liste):
    vmax, imax = liste[0], 0
    for i in range(1, len(liste)):
        if liste[i] > vmax:      # strict : on garde le premier indice du maximum
```

```
            vmax, imax = liste[i], i
    return (vmax, imax)
```

L'inégalité stricte garantit que l'on renvoie le **plus petit** indice réalisant le maximum. On suppose la liste non vide.

Question 19

Étape 1. Q19. La stratégie gloutonne

Remarque sur le code fourni : dans `initialiserGlouton`, il faut lire `E[Obs[0]][i]` (ligne de l'observation, colonne du symbole) ; `E[Obs[0][i]]` est une coquille.

```
def glouton(Obs, P, E, K, N):
    etats = [initialiserGlouton(Obs, E, K)]    # meilleur premier symbole
    for j in range(1, N):
        i = etats[-1]                           # symbole courant
        probas = [E[Obs[j]][k] * P[i][k] for k in range(K)]  # poids des K arcs sortants
        p, k = maximumListe(probas)             # arc le plus probable
        etats.append(k)
    return etats
```

Question 20

Étape 1. Q20. Complexité du glouton

L'initialisation examine K valeurs. Puis $N-1$ tours de boucle ; à chaque tour, on calcule K produits et on cherche le maximum de K valeurs : $O(K)$.

Complexité : $O(NK)$.

Question 21

Étape 1. Q21. Le glouton n'est pas optimal

Chemin glouton : depuis σ, l'arc le plus lourd mène au symbole 0 ($0{,}6 > 0{,}4$) ; depuis ce sommet, l'arc le plus lourd reste sur 0 ($0{,}5 > 0{,}1$). Chemin $\sigma \to 0 \to 0 \to \tau$, de probabilité $0{,}6 \times 0{,}5 \times 1 = 0{,}30$.

Meilleur chemin : $\sigma \to 1 \to 0 \to \tau$, de probabilité $0{,}4 \times 0{,}9 \times 1 = 0{,}36 > 0{,}30$. (Les deux autres valent $0{,}6 \times 0{,}1 = 0{,}06$ et $0{,}4 \times 0{,}01 = 0{,}004$.)

Le glouton renvoie $[0, 0]$ alors que l'optimum est $[1, 0]$: l'approche gloutonne n'est **pas optimale**. Un premier choix un peu moins bon pouvait ouvrir un arc bien meilleur.

Question 22

Étape 1. Q22. Se ramener à un plus court chemin

On remplace chaque poids $p \in]0,1]$ par $w = -\ln p \geqslant 0$. Pour un chemin de poids $p_1, \dots, p_r$:

$$\sum_{\ell} w_\ell = -\ln\Big(\prod_{\ell} p_\ell\Big).$$

Comme $-\ln$ est strictement décroissante, maximiser le produit des probabilités revient à **minimiser la somme des poids** w, qui sont positifs (un arc de probabilité nulle est simplement retiré). On cherche donc un plus court chemin de σ à τ dans un graphe à poids positifs : on peut utiliser l'**algorithme de Dijkstra**.

(Le graphe étant sans circuit et rangé en couches, une simple programmation dynamique couche par couche suffit : c'est l'algorithme de Viterbi qui suit.)

Question 23

Étape 1. Q23. Construire les tableaux de Viterbi

Ordre de calcul : la relation donne la colonne j à partir de la colonne $j-1$ seulement. On remplit donc les colonnes de gauche à droite (de bas en haut au sens de la programmation dynamique), la colonne 0 étant déjà remplie par `initialiserViterbi`.

```
def construireTableauViterbi(Obs, P, E, K, N):
    T, argT = initialiserViterbi(E, Obs[0], K, N)
    for j in range(1, N):            # colonne j, à partir de la colonne j-1
        for i in range(K):           # symbole i à l'instant j
            valeurs = [T[k][j-1] * P[k][i] * E[Obs[j]][i] for k in range(K)]
            T[i][j], argT[i][j] = maximumListe(valeurs)   # meilleur prédécesseur k
    return T, argT
```

Question 24

Étape 1. Q24. Lire la séquence la plus probable

Dernière colonne ($j = 7$) de `T` : $1{,}8 \cdot 10^{-5}$, $3{,}5 \cdot 10^{-6}$, $1{,}3 \cdot 10^{-5}$. Le maximum est en ligne 0 : le dernier symbole est 0.

Remontée : `argT[i][j]` donne le symbole à l'instant $j-1$.

- $j = 7$, symbole 0 : `argT[0][7]` $= 0$, donc symbole 0 en $j = 6$;
- $j = 6$, symbole 0 : `argT[0][6]` $= 1$, donc 1 en $j = 5$;
- $j = 5$, symbole 1 : `argT[1][5]` $= 1$, donc 1 en $j = 4$;
- $j = 4$, symbole 1 : `argT[1][4]` $= 2$, donc 2 en $j = 3$;
- $j = 3$, symbole 2 : `argT[2][3]` $= 0$, donc 0 en $j = 2$;

- $j = 2$, symbole 0 : `argT[0][2]` $= 0$, donc 0 en $j = 1$;
- $j = 1$, symbole 0 : `argT[0][1]` $= 2$, donc 2 en $j = 0$ (et `argT[2][0]` $= -1$: on est arrivé à la source).

Séquence la plus probable : $[2, 0, 0, 2, 1, 1, 0, 0]$.

Question 25

Étape 1. Q25. Complexités de Viterbi

Temps : il y a $(N-1)K$ cases à calculer ; chacune demande K produits et un maximum sur K valeurs, soit $O(K)$. Avec l'initialisation en $O(NK)$:

$$\text{temps} : O(NK^2), \qquad \text{espace} : O(NK) \text{ (les deux tableaux } K \times N).$$

C'est polynomial, à comparer aux K^N chemins de Q17.

Le piège

En Q24, lire `argT` dans le mauvais sens : `argT[i][j]` est le symbole à l'instant $j-1$, pas à l'instant $j+1$.

La méthode

Chemin optimal dans un graphe en couches : programmation dynamique couche par couche (valeur optimale + prédécesseur), puis remontée depuis la meilleure case de la dernière couche.

Rappels de cours utilisés

- **Algorithme glouton.** À chaque étape, il fait le choix qui paraît le meilleur sur le moment, sans jamais revenir en arrière. Il est rapide, mais pas toujours optimal : un contre-exemple suffit à le prouver.
- **Plus court chemin.** Dans un graphe à poids positifs, l'algorithme de Dijkstra calcule les plus courts chemins depuis une source. Le logarithme transforme un produit en somme : $\ln(ab) = \ln a + \ln b$, et $-\ln p \geqslant 0$ quand $0 < p \leqslant 1$.
- **Programmation dynamique.** On écrit une relation de récurrence entre sous-problèmes, on remplit un tableau dans l'ordre où les valeurs nécessaires sont déjà calculées (de bas en haut), et on garde de quoi reconstruire la solution (les « prédécesseurs »).